

Django Module 1 Important

1. Compare MVC with MVT Architecture (6 Marks)

MVC (Model-View-Controller)	MVT (Model-View-Template)
MVC stands for Model, View, Controller .	MVT stands for Model, View, Template .
Commonly used in many web frameworks.	Mainly used in Django.
Controller handles user requests and application flow.	Django framework itself performs most controller-like functions through URL routing and framework internals.
View is responsible for presentation (UI).	Template is responsible for presentation (UI).
Model manages data and database operations.	Model manages data and database operations.
Controller interacts with Model and View.	View interacts with Model and Template.
User Request → Controller → Model/View → Response	User Request → URL → View → Model → Template → Response
More responsibility is placed on the developer to manage controllers.	Simplifies development because Django handles much of the control logic internally.

MVC Architecture Components

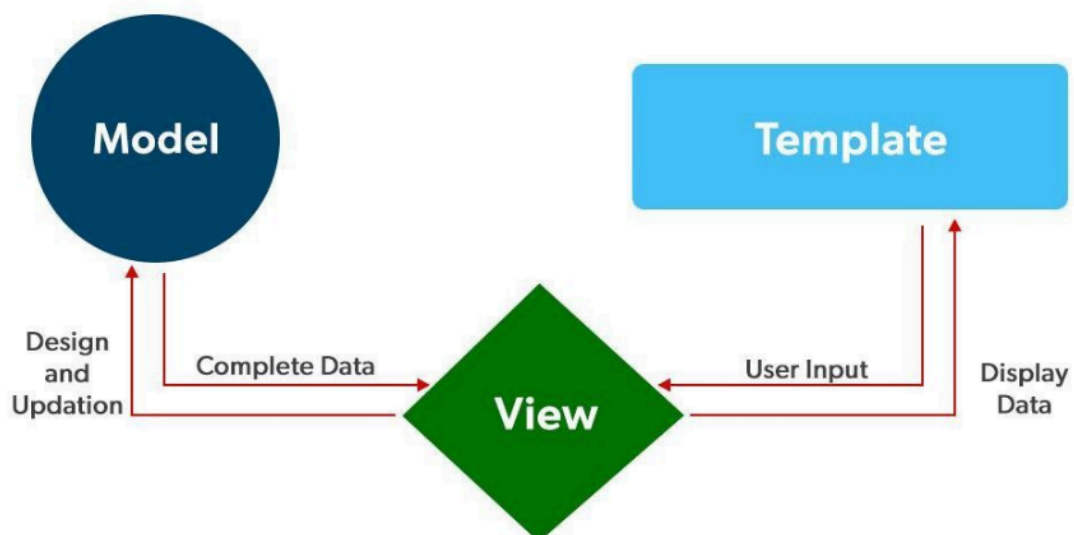
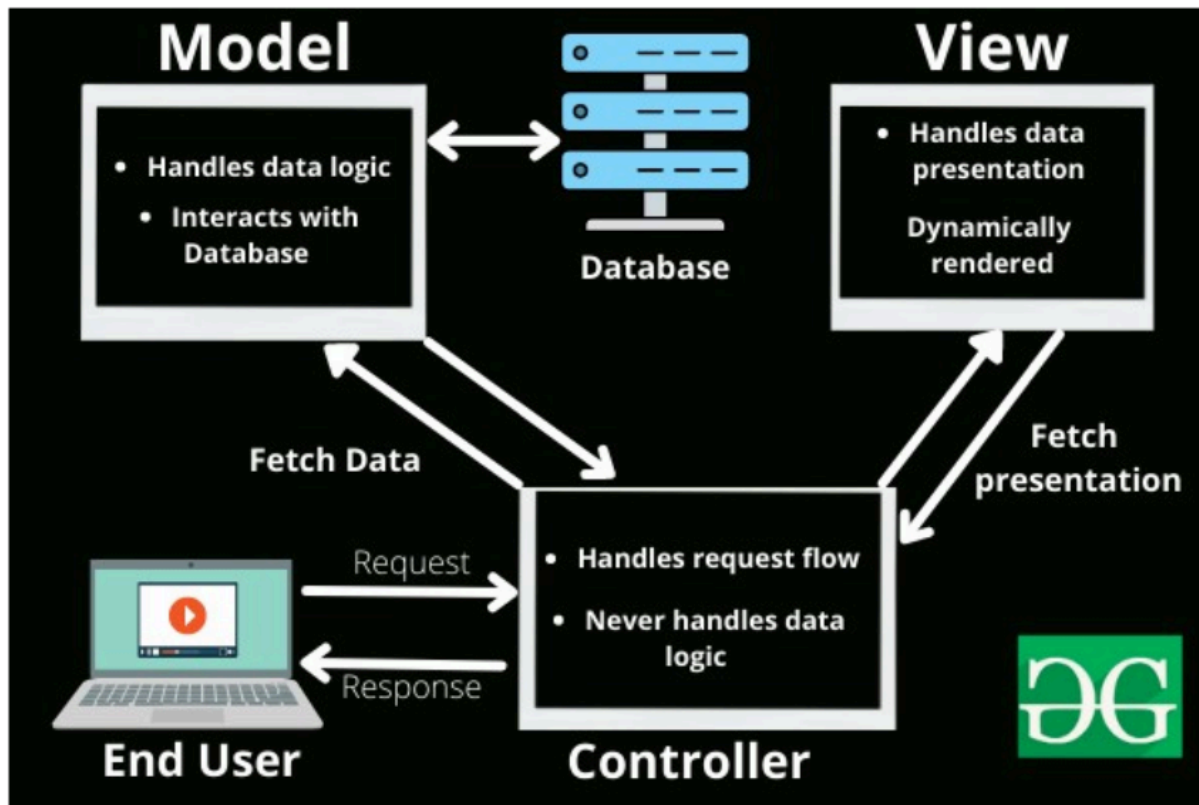
1. **Model** – Handles data and database operations.
2. **View** – Displays data to the user.
3. **Controller** – Processes requests and controls application flow.

MVT Architecture Components

1. **Model** – Handles database and data structure.
2. **View** – Contains business logic and processes user requests.
3. **Template** – Handles presentation layer (HTML).

Conclusion

MVC and MVT both separate business logic from presentation logic. The main difference is that MVC uses a **Controller**, whereas Django's MVT replaces it with a **View**, while the framework itself handles controller-like responsibilities internally. This makes Django development faster and easier.



2. Outline the Steps to Install and Set Up a Django Project and App

Step 1: Install Python

Download and install Python.

Verify installation:

```
python --version
```

Step 2: Create a Virtual Environment

Move to the project directory:

```
cd Desktop
```

Create virtual environment:

```
python -m venv env
```

Activate virtual environment:

```
env\Scripts\activate
```

After activation, **(env)** appears in the terminal.

Step 3: Install Django

```
pip install django
```

Check Django version:

```
django-admin --version
```

Step 4: Create a Django Project

```
django-admin startproject myproject
```

Move into the project folder:

```
cd myproject
```

Step 5: Run the Development Server

```
python manage.py runserver
```

Open the browser and visit:

`http://127.0.0.1:8000/`

The Django welcome page will be displayed.

Step 6: Create a Django App

`python manage.py startapp myapp`

Add the app name in `settings.py`:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'myapp',  
]
```

Necessary Commands

Command	Purpose
<code>python --version</code>	Check Python installation
<code>python -m venv env</code>	Create virtual environment
<code>env\Scripts\activate</code>	Activate virtual environment
<code>pip install django</code>	Install Django
<code>django-admin --version</code>	Check Django version
<code>django-admin startproject myproject</code>	Create Django project
<code>python manage.py runserver</code>	Start development server
<code>python manage.py startapp myapp</code>	Create Django app

3. Creating Views and Mapping URLs with Code Snippets

Creating Views

A view is a Python function that handles HTTP requests and returns HTTP responses. Views are created inside the `views.py` file of an app.

Example:

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to Django")

def about(request):
    return HttpResponse("This is About Page")
```

- `request` represents the user request.
- `HttpResponse()` sends a response to the browser.

Example using a template:

```
from django.shortcuts import render

def home(request):
    return render(request, 'home.html')
```

Responsibilities of Views:

- Receives HTTP requests
- Interacts with models
- Sends data to templates
- Returns HTTP responses

Mapping URLs

URL mapping connects a URL path to a specific view function. It is done in `urls.py`.

Project-Level URL Mapping

File: `project/urls.py`

```
from django.contrib import admin
from django.urls import path
from myapp import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", views.home),
    path('about/', views.about),
]
```

- `' '` maps to the home page.
- `'about/'` maps to the about page.
- `views.home` calls the `home()` function.
- `views.about` calls the `about()` function.

App-Level URL Mapping

Step 1: Create `urls.py` inside the app

```
from django.urls import path
from . import views
```

```
urlpatterns = [
    path("", views.home),
    path('about/', views.about),
]
```

Step 2: Include App URLs in Project `urls.py`

```
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('myapp.urls')),
]
```

- `include()` links app-level URLs with project-level URLs.
- Improves modularity and organization of URL routing.

4. HTTP Request–Response Cycle in Django

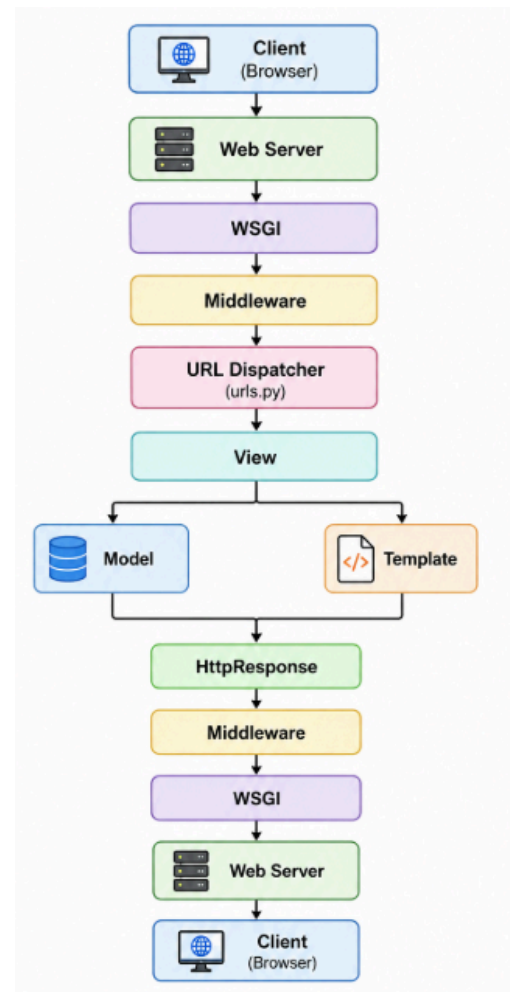
Request Phase (Client to View)

1. **Client Request**
 - User enters a URL, clicks a link, or submits a form.
 - Browser creates an `HttpRequest` object.
2. **Web Server and WSGI**
 - Web server receives the request.
 - WSGI acts as a bridge between the web server and Django.
3. **Middleware Processing**

- Request passes through middleware components.
 - Middleware performs tasks such as authentication, security checks, and session handling.
4. **URL Dispatcher**
- Django checks `urls.py`.
 - Matches the URL with the appropriate view.
 - If no match is found, a 404 error is returned.
5. **View Processing**
- The matched view function is executed.
 - View can:
 - Access request data
 - Interact with models/database
 - Perform business logic
 - Send data to templates

Response Phase (View to Client)

1. **Response Generation**
 - View returns an `HttpResponse`.
 - Response may be:
 - HTML page
 - JSON data
 - Redirect
 - Plain text
2. **Middleware Processing**
 - Response travels through middleware in reverse order.
 - Middleware may add headers, logging, or security features.
3. **WSGI and Web Server**
 - Response is passed back through WSGI to the web server.
4. **Client Display**
 - Browser receives the response.
 - HTML is rendered and displayed to the user.

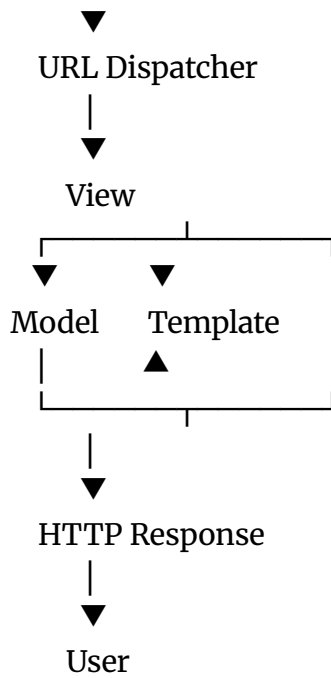


5. Django Architecture (MVT Architecture)

Django follows the **MVT (Model–View–Template)** architectural pattern. It separates an application into three main components: **Model, View, and Template**. This separation makes applications organized, scalable, and easy to maintain.

Neat Diagram

User Request
|



Components of MVT

1. Model

The Model handles all data-related logic of the application. It defines database tables and manages interactions with the database using Django ORM.

Responsibilities:

- Managing data (CRUD operations)
- Defining database structure
- Enforcing business rules
- Communicating with the database

Example:

In a bookstore application, the Model handles book details such as title, author, price, and stock quantity.

2. View

The View contains the business logic and handles HTTP requests and responses. It receives user requests, interacts with the Model, and sends data to the Template.

Responsibilities:

- Receiving and processing user requests
- Interacting with the Model
- Sending data to the Template

- Returning HTTP responses

Example:

In a bookstore application, the View handles searching books, adding books to cart, and displaying book details.

3. Template

The Template is responsible for the presentation layer (UI). It defines how data is displayed to the user using HTML and Django Template Language.

Responsibilities:

- Rendering data in HTML format
- Displaying user interface elements
- Presenting dynamic content

Example:

In a bookstore application, the Template displays the list of books, search forms, and book details pages.

Working of Django Architecture

1. User sends a request through the browser.
2. URL Dispatcher (`urls.py`) maps the URL to the appropriate View.
3. View processes the request.
4. If required, View interacts with the Model to retrieve or update data.
5. View sends data to the Template.
6. Template generates the HTML page.
7. Django returns the HTTP response to the user.

Features of MVT

- Clear separation of business logic, UI logic, and request handling.
- Built-in URL routing system.
- Automatic admin interface.
- Powerful ORM for database operations.
- Supports rapid development.
- Secure and scalable architecture.

6.Django Project Structure

When a Django project is created using:

```
django-admin startproject myproject
```

The structure is:

```
myproject/
|
|— manage.py
|
|— myproject/
|   |— __init__.py
|   |— settings.py
|   |— urls.py
|   |— asgi.py
|   |— wsgi.py
```

I) settings.py

`settings.py` is the main configuration file of a Django project. It controls how the project behaves.

Important Sections

DEBUG

`DEBUG = True`

- Used during development.
- Shows detailed error pages.
- Should be `False` in production.

ALLOWED_HOSTS

`ALLOWED_HOSTS = []`

- Specifies which domains can access the project.
- Required when `DEBUG = False`.

INSTALLED_APPS

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'myapp',
]
```

- List of apps used in the project.
- Newly created apps must be added here.

DATABASES

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

- Defines database configuration.
- Django uses SQLite by default.

MIDDLEWARE

- Handles request and response processing.
- Used for security, sessions, authentication, etc.

TEMPLATES

- Defines template folder location.
- Controls HTML rendering settings.

STATICFILES

- Used for CSS, JavaScript, and images.
- Configures static file handling.

II) urls.py

`urls.py` is used for URL routing. It connects URLs to views.

Example:

```
from django.contrib import admin
from django.urls import path
from myapp import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", views.home),
]
```

Purpose:

- Maps URL patterns to view functions.
- Django checks this file when a user enters a URL.
- Directs the request to the correct view.

III) views.py

`views.py` contains the business logic of the application.

It handles user requests, processes data, interacts with models, and returns responses.

Example:

```
from django.http import HttpResponse
from django.shortcuts import render

def home(request):
    return HttpResponse("Welcome to Django")

def about(request):
    return render(request, 'about.html')
```

Responsibilities:

- Receives HTTP requests.
- Interacts with models.
- Sends data to templates.
- Returns HTTP responses.

7. I) Virtual Environment

A virtual environment is an isolated Python environment created for a specific project. It contains its own:

- Python interpreter
- Installed packages
- Django version and dependencies

Each Django project can have its own virtual environment independent of other projects and the system Python.

Why Virtual Environments are Needed

- Avoids version conflicts between projects.
- Keeps project dependencies isolated.
- Makes projects easier to maintain and deploy.
- Helps in team collaboration by ensuring consistent environments.

Creating a Virtual Environment

```
python -m venv venv
```

Activating the Virtual Environment

```
venv\Scripts\activate
```

After activation, (venv) appears in the terminal.

Deactivating the Virtual Environment

```
deactivate
```

II) pip

pip is Python's default package manager used to install, upgrade, and remove packages.

Uses of pip

- Install packages
- Upgrade packages
- Remove packages
- Manage project dependencies

In Django, pip is used to install Django and other required libraries inside the virtual environment.

Installing Django Using pip

```
pip install django
```

Check Django Version

```
django-admin --version
```

View Installed Packages

```
pip list
```

Creating requirements.txt

pip freeze > requirements.txt

Example:

```
Django==4.2.7
asgiref==3.7.2
sqlparse==0.4.4
```

Installing Dependencies from requirements.txt

pip install -r requirements.txt

Importance of pip

- Ensures project-specific dependency management.
- Prevents conflicts between Django projects.
- Makes deployment and testing easier.
- Supports reproducibility across development environments.

8. Core Principle of Loose Coupling in Django

Loose coupling is a core design philosophy in Django where different components of the framework are designed to be as independent as possible, with minimal dependency on each other.

The main goal of loose coupling is to improve:

- Flexibility
- Maintainability
- Testability
- Scalability

In a loosely coupled system, changes in one component have little or no effect on other components.

Key Manifestations of Loose Coupling in Django

1. MVT Architecture

- Separates business logic, data handling, and presentation.
- Templates do not need to know how data is stored in the database.
- Models, Views, and Templates work independently.

2. URL Configuration (URLconfs)

- URL definitions are separate from view functions.
- URL paths can be changed without modifying view logic.

3. Interchangeable Components

- Django allows developers to use alternative tools such as different template engines or database layers with minimal changes.

4. Signals

- Signals notify applications when specific actions occur (e.g., saving a model).
- The sender does not need to know which receivers are listening.

5. Request and Response Objects

- Views receive an `HttpRequest` object and return an `HttpResponse` object.
- Views do not need to know how requests are processed internally.

Benefits of Loose Coupling

- **Easier Maintenance:** Bugs can be fixed with less impact on other parts of the application.
- **Increased Flexibility:** Applications can adapt easily to changing requirements.
- **Improved Testability:** Components can be tested independently.
- **Better Scalability:** Individual components can be scaled or modified separately.